

MatchaLog Product Requirements

Version 1.1 | September 5, 2026 | Product scope and release requirements

1 Product purpose and release scope

MatchaLog helps people remember what matcha they have tried, manage what they have, and decide what to try next. The first release combines a curated product catalog, a private personal collection, and public community reviews.

The product remains a responsive web application using Next.js App Router, React, JavaScript, and Supabase. Existing implementation and design conventions should be preserved unless a requirement below calls for a change.

Primary users

The Casual Sipper needs quick saves and a simple way to remember past purchases. The Matcha Enthusiast needs collection statuses, useful product information, and reviews that explain preparation context. Recipe creators remain a future audience rather than a separate MVP workflow.

Release priorities

Release	Included scope
MVP	Curated catalog, product search and details, authentication, private stash, public reviews, basic profile/settings, and catalog suggestion and moderation tools.
Next	Public profile discovery, optional collection sharing, follows, and an activity feed with explicit privacy rules.
Later	Recipe creation, recipe detail, and community recipe discovery, subject to evidence of user demand.

Changes from version 1.0

Authentication and ownership controls precede multi-user writes. Stash statuses now represent ownership and use. Catalog operations, product identity, review context, privacy, product-value metrics, and testable acceptance criteria are explicit. Accessibility, responsive behavior, and error recovery apply to every phase.

The previous document described Discover and basic Product Detail as built or in progress. Those are historical status notes; completion of this revision’s requirements must be verified against the implementation before release.

2 Catalog and discovery

Catalog ownership and missing products

MVP catalog data is manually curated. Only an authorized catalog administrator may publish or edit canonical products. A signed-in user can suggest a missing product or report a correction; submission does not publish catalog content automatically.

Suggestions require a product name and brand; a source link and notes are optional. A user may mark a suggestion as Want to Try. It appears in a separate pending section of their stash and becomes a normal entry when linked to an approved product. Rejection includes a reason; duplicate suggestions link to the existing product without duplicating stash entries.

The curator workflow supports pending, approved, rejected, and linked-to-existing outcomes. Before launch, assign an owner for initial catalog population, suggestion triage, product corrections, and reported reviews. Define a realistic review cadence and display submission status to the requester.

Product identity and fields

A canonical product represents a named matcha from one brand. Package sizes share the product page and reviews. Use a separate product only when the producer markets a materially distinct product, not merely a different tin size. Harvest or batch information is optional; a new harvest alone does not create a new product in MVP.

Required catalog fields are ID, name, brand, publication status, and created/updated timestamps. Origin, description, image, source link, producer-described grade, flavor notes, and harvest details are optional and must be shown as unknown when absent. Describe flavor notes as producer or curator information, separate from user reviews.

Package options contain weight in grams and, when available, price amount, currency, source URL, and date checked. Prices are reference information, not live offers. Price filtering is deferred until coverage is sufficient; any future comparison must use a common weight and currency policy.

Discover and product detail

Discover at / searches product name and brand with the established 300 ms debounce. Provide pagination, origin and available grade filters, and sorts for newest, name, and community rating. Unknown metadata remains browsable. Preserve search/filter state on return from a product page.

Cards show image or fallback, name, brand, origin when known, average rating and review count, and a stash action. Product pages at /products/[productId] show the full product data, available packages, dated prices, review summary/list, stash controls, and navigation back to discovery. Unrated products display No reviews, never a zero-star rating.

3 Authentication and private collections

Authentication and access

Anyone may browse published products and public reviews. Saving, reviewing, suggesting products, and editing settings require a signed-in user. A login prompt preserves the intended action and returns the user to its context after authentication; the user can then complete the action.

MVP supports email/password registration, login, logout, password reset, and verification handling. Google sign-in is optional after the core flow. Validate sessions on protected writes and enforce record ownership in the database. Enable row-level security before any real multi-user data is accepted. Restrict catalog writes to authorized administrators.

A single-user prototype may use an isolated development environment with disposable data. World-writable production data and hardcoded production user identities are not permitted. Authentication is a prerequisite for shared stash and review work, not a later retrofit.

Stash behavior

The stash at /stash is private by default and in MVP. Each user has one entry per canonical product, representing current collection state rather than individual tins or lots. Multiple packages, quantities, and purchase history are outside MVP.

Status	Meaning
Want to Try	Interested in trying or buying; ownership is not implied.
Unopened	Owned, but not currently opened.
Open	Currently using the product.
Finished	No current supply remains after use.

A quick save defaults to Want to Try and offers a status change. Users may move directly between any statuses, including Finished back to Unopened or Open for repeat purchases. These are useful states, not a mandatory sequence.

Store `tried_before` separately from status. Moving to Open or Finished marks it true; users can also set or correct it manually. Returning to Want to Try or Unopened preserves it. Review publication also marks an existing stash entry as tried before, but never creates a stash entry automatically.

Stash supports filtering by status and tried-before value, and sorting by date added, product name, or My rating. My rating uses the owner's review; unrated items appear last. Empty collections link to Discover and the missing-product suggestion flow.

Removal deletes only the collection entry, including its local tried-before value. It does not delete the user's review or the catalog product. Confirm this distinction in the removal interaction and provide a clear recovery path for accidental removal.

4 Reviews profiles and privacy

Public reviews

A review requires a whole-number rating from 1 to 5. Text is optional, with a maximum of 1000 characters. Prepared as is optional, with values Straight matcha, Latte, or Cooking or baking. One editable review is allowed per user and product.

No verified purchase or stash membership is required. The form asks users to review products they have tried and clearly labels publication as public. Publication does not expose private collection details. The reviewer may edit or delete their own review; the form preserves input on validation or network failure.

Show the author's public display name and avatar, rating, optional preparation context, text, and posting date; label edited reviews. Support newest, highest-rating, and lowest-rating sorts with pagination. Aggregate only currently published, non-removed reviews and update the average/count after creation, rating edits, deletion, or moderation.

Profile and settings

MVP /profile provides the signed-in user's display name, avatar, optional bio, account settings, and their reviews. Display names and avatars appear publicly with reviews; email addresses never do. Other users cannot browse private stash records through profile pages or APIs.

Settings explain what is public and private, support profile edits, and provide an account-deletion flow. Account deletion removes the user's stash, reviews, profile, pending suggestions, and owned media; canonical catalog records remain. Any retained operational data and retention period must be documented before launch.

Moderation and uploads

Users can report inappropriate reviews. An authorized moderator can inspect reports and hide or restore a review with a recorded reason; hidden reviews are excluded from public lists and rating summaries. Profile/avatar reports use the same moderation queue. Define support ownership before release.

Avatar and catalog image uploads require file type and size validation, access controls, descriptive alternatives or appropriate decorative treatment, and a visible error state. Accept JPEG, PNG, or WebP images up to 5 MB for MVP. Store media references so replaced or deleted files can be cleaned up.

Collection sharing and activity privacy

Private stash actions produce no public activity in MVP. Future collection sharing must be an explicit opt-in. Private records must remain inaccessible even when a visitor knows an ID or URL. Making a collection private removes its shared visibility and associated stash activity; source deletion or moderation also removes the corresponding public activity.

5 Data and interface requirements

This section defines required data relationships and invariants. Database migrations and detailed access policies are implementation deliverables. All user-owned writes use the authenticated identity, never a client-supplied owner ID.

Entity	Required structure and constraints
matcha_products	Canonical catalog fields from section 2; publication status; admin-controlled writes. Preserve existing IDs when curating records.
product_packages	Product reference, positive weight_grams, optional nonnegative price and currency, price source and checked date. Required price metadata must accompany a displayed price.
profiles	One row per auth user; display_name, avatar reference, optional bio, timestamps. Separate public fields from account/private data.
user_stash	ID, user_id, product_id, status, tried_before, created_at, updated_at. Unique user/product pair; constrain statuses to the four defined values.
reviews	ID, user_id, product_id, rating, optional review_text and prepared_as, visibility/moderation state, timestamps. Unique user/product pair; enforce rating and text limits.
product_suggestions	Requester, proposed name/brand, optional source/notes, want-to-try intent, review status/reason, resolved product reference, timestamps. Private to requester and curator.
content_reports	Reporter, target type and ID, reason, triage status, moderator outcome and timestamps. Restrict access to reporter submission and authorized moderation.

API behavior

Retain Next.js Route Handlers under app/api. GET /api/stash lists the current user's entries; POST /api/stash saves a product idempotently; PATCH /api/stash/[id] changes state; DELETE removes the owned entry. Repeated saves must not reset an existing status.

GET /api/products/[productId]/reviews lists public reviews. POST /api/reviews creates or updates the caller's review; DELETE /api/reviews/[id] deletes only their review. Provide authenticated suggestion submission/status routes and protected curator/moderator actions. Paginate catalog, stash, and review lists with stable ordering.

Validate identifiers, allowed values, text lengths, and ownership before mutation. Return understandable unauthenticated, forbidden, invalid-input, not-found, and transient-failure states. Ensure duplicate/retried requests cannot create duplicate stash or review rows. Future follows require a unique follower/following pair and prohibit self-following.

6 Success measures and release acceptance

Product value

Collect an initial four-week baseline after beta launch, then set numerical product targets and review them monthly. Define events consistently, exclude staff/test traffic, and avoid sending review text or private stash contents to analytics.

Measure	Definition
First-session activation	Share of new signed-in users who save at least one catalog product during their first authenticated session. A session ends after 30 minutes of inactivity.
Collection return rate	Share of newly activated users who return on a later calendar day within 14 days to add a product or change collection status. Count only fully observed cohorts.
Repeat reviewing	Share of first-time reviewers who publish a review of a second product within 30 days. Exclude edits; use fully observed cohorts.
Review completion	Successful review publications divided by sessions in which the user starts entering review data, measured weekly. Initial operational target exceeds 80%.

Reliability and performance

Target greater than 99% successful valid stash mutations, measured weekly; count failed valid requests, but exclude deliberate validation/auth rejections. Instrument catalog and product-page load performance under a documented mobile test profile and set a release budget before beta. Search-to-click time is diagnostic, not a stand-alone success target: longer comparison can reflect useful browsing.

Replace the ambiguous Lighthouse mobile usability score with separate performance and accessibility checks, plus manual keyboard and mobile testing. Future feed performance receives its own budget when the social phase is scoped.

Acceptance criteria for MVP

Discovery: name and brand searches return matching published products; filters combine correctly; empty/error states offer recovery; pagination has stable ordering; unrated items show No reviews.

Collections: repeated saves yield one entry; every status transition and repeat-purchase transition works; tried-before history survives ordinary status changes; private entries cannot be read or changed by another user; removal leaves reviews intact.

Reviews: a second submission updates the existing review; invalid ratings/text are rejected; failed submissions preserve input; edits/deletion/moderation update public counts and averages; unauthorized edits fail.

Catalog operations: suggestions retain requester intent, approved duplicates link to one canonical product, and rejected suggestions show a reason. Pricing is hidden when required context is missing.

Usability: core flows work at narrow phone widths and with keyboard navigation; fields have labels, focus is visible, errors are announced, images have appropriate alternatives, and reduced-motion preferences are respected. Loading and retry states are present in every release flow.

7 Delivery plan and future features

Implementation sequence

Stage	Scope and exit condition
1 Foundation	Confirm existing behavior; establish auth, database ownership controls, profile records, catalog curation, and an initial published dataset. No production anonymous writes.
2 Core MVP	Complete discovery, product details, private collections, reviews, suggestions, basic settings, and moderation. Meet section 6 acceptance criteria throughout.
3 Beta validation	Instrument defined metrics, verify access boundaries and deletion behavior, test core mobile/keyboard flows, assign operational owners, and gather the baseline.
4 Social follow-up	Proceed after reviewing beta usage and demand. Add profile discovery, opt-in collection sharing, follows, and a privacy-aware activity feed.
5 Recipe follow-up	Proceed if demand warrants it. Scope publishing, editing, moderation, images, and discovery before estimating delivery.

Estimate delivery after confirming implementation status, staffing, and acceptance scope. The version 1.0 week estimates are superseded; this sequence expresses dependencies rather than an unvalidated schedule.

Future social requirements

Profiles may expose public reviews, recipes, follow counts, and explicitly shared collections. The feed may include public review publication, shared stash additions, published recipes, and follows. Do not emit new publication events for routine edits or duplicate retries. Define whether following itself is public before launch.

Each activity event must identify its source type and record, honor current source visibility, and disappear when the source is deleted or hidden. Database triggers are an implementation option, not a reason to publish private actions. Start with paginated refresh-based loading; realtime delivery requires a demonstrated need.

Future recipe requirements

Recipes support a title, description, ingredient amounts/units/items, ordered steps, image, optional linked product, and author. Prep time and category are required if shown on cards. Define draft versus published states, edit/delete behavior, moderation, and upload rules before implementation. Only published recipes may appear in discovery or activity.

8 Technical conventions and decision register

Implementation conventions

Keep app/ for App Router pages and layouts, app/api/ for Route Handlers, components/ for reusable UI, and lib/ for shared utilities. Prefer Server Components for initial reads and use client components for interaction. Separate browser and server database clients as needed for session and permission handling; never expose privileged credentials to the browser.

Preserve established styling: matcha green #4CAF50, light mint #edfff0, system-ui typography, and existing card/control radii where usable. Color contrast, visible focus, legibility, and touch usability take priority over literal values. Hover scaling is optional, must not shift layout, and should be disabled for reduced-motion users.

Retain the established 300 ms search debounce. Use Supabase Storage for images with explicit upload ownership and cleanup behavior. Introduce SWR or React Query only when interaction and cache consistency warrant it. Verify deployed framework versions and dependencies during implementation rather than treating the original Next.js 14 label as a required version.

Decisions resolved by this revision

MVP product data is curated with user suggestions. Reviews require authentication and a self-reported tasting experience, but no purchase verification or stash entry. Collections are private in MVP. Authentication precedes multi-user writes. Recipes and social features are later releases. Package size is separate from canonical product identity. Collection status and prior tasting are distinct.

Remaining decisions and owners

Decision	Owner and timing
Initial catalog breadth, curation cadence, and named support/moderation staff	Product / Operations, before beta.
Mobile performance test profile and release budget	Engineering, before beta acceptance.
Operational data retention and account deletion completion policy	Product / Engineering, before launch.
Numerical activation, return, and repeat-review targets	Product, after the first four-week baseline; use mature cohorts.
Public follow visibility and collection-sharing controls	Product, before the social phase.
Recipe publication/moderation policy and release decision	Product, before the recipe phase.

Out of scope for MVP

Native mobile apps, commerce or vendor storefronts, AI recommendations, multilingual support, push notifications, offline mode, multiple-tin inventory and purchase history, live prices or currency conversion, social feeds/follows, and recipe publishing. These exclusions do not prevent basic responsive web access or manual catalog correction.